

Lecture 12: Mid-Semester Review

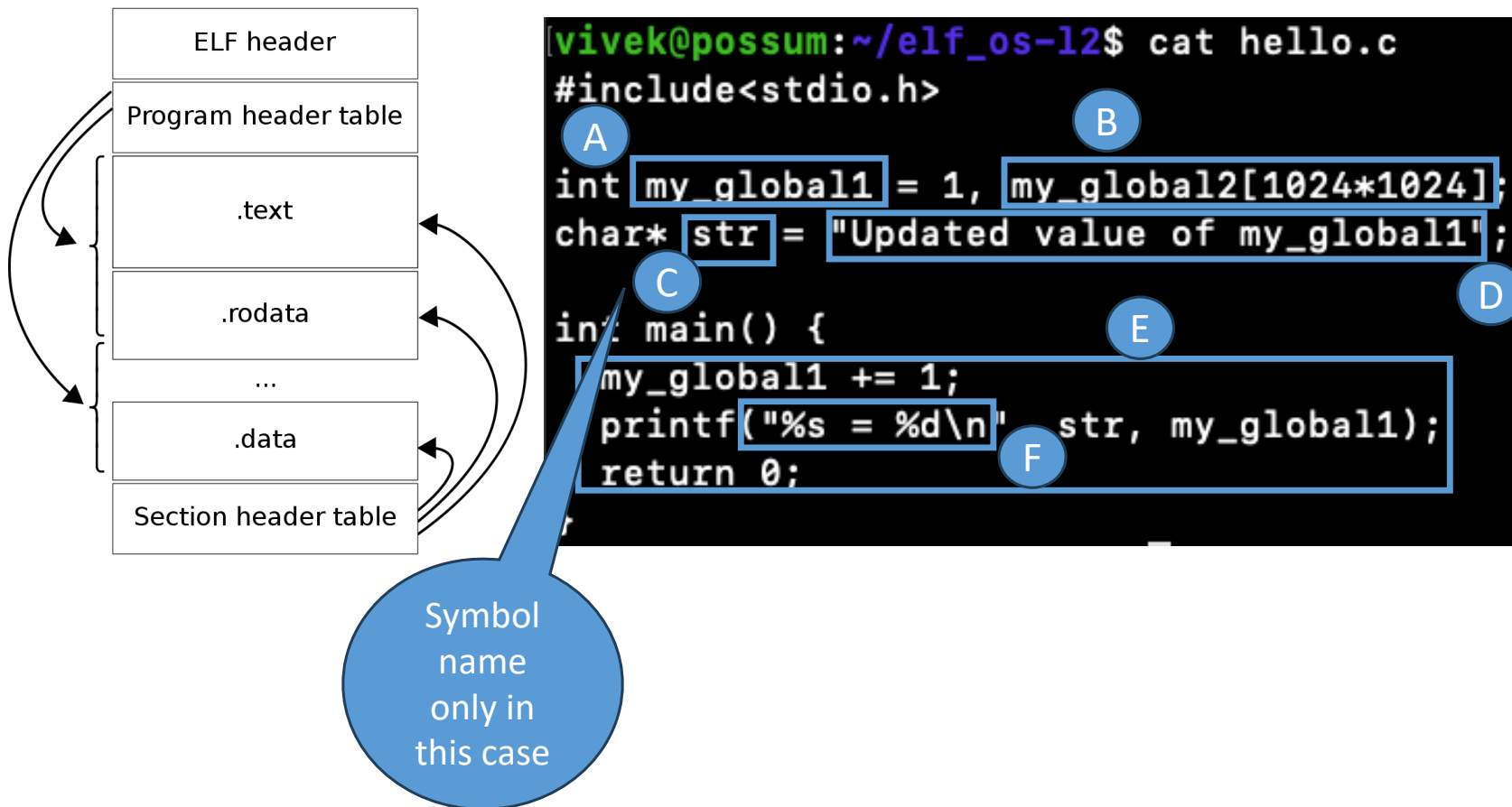
Vivek Kumar

Computer Science and Engineering

IIIT Delhi

vivekk@iiitd.ac.in

Executable and Linkable Format (ELF)



1. .bss
2. .strtab
3. .rodata
4. .text
5. .data
6. .symtab

Program Execution (1/6)

```
L1: int g=0;
```

```
L2: void main() {
```

```
L3:   int *a = (int*) malloc(4);
```

```
L4:   char *b = "Hello World";
```

```
L5:   foo(a);
```

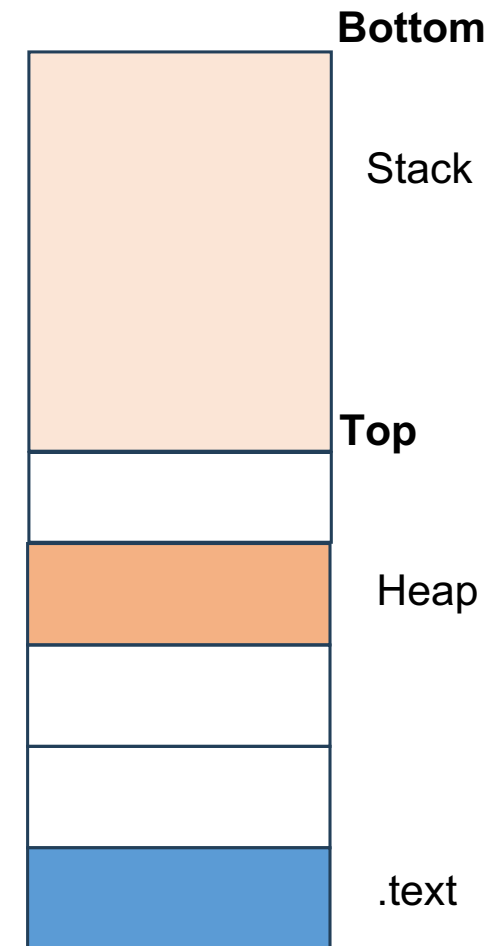
```
L6:   g=*a;
```

```
L7: }
```

```
L8: void foo(int* c) {
```

```
L9:   *c = 20;
```

```
L10:}
```



Program Execution (2/6)

L1: int g=0;

L2: void **main**() {

L3: int *a = (int*) malloc(4);

L4: char *b = "Hello World";

L5: **foo**(a);

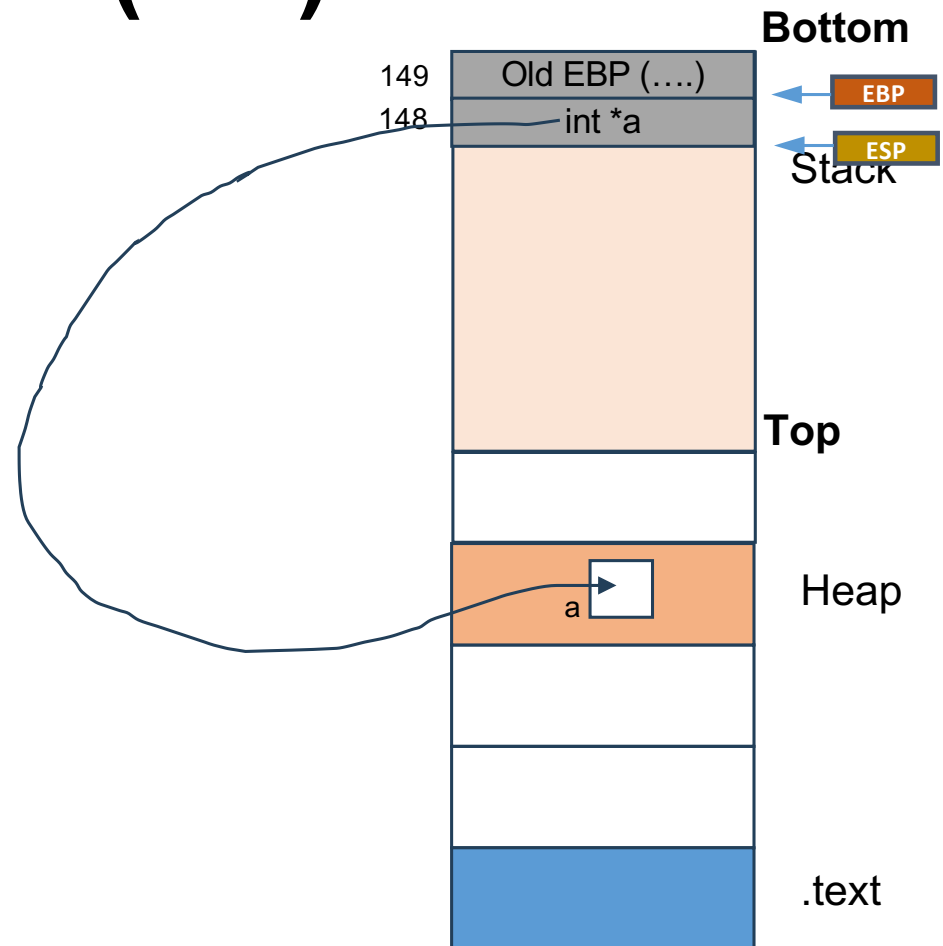
L6: g=*a;

L7: }

L8: void **foo**(int* c) {

L9: *c = 20;

L10: }



Program Execution (3/6)

L1: int g=0;

L2: void **main**() {

L3: int *a = (int*) malloc(4);

L4: char *b = "Hello World";

L5: **foo**(a);

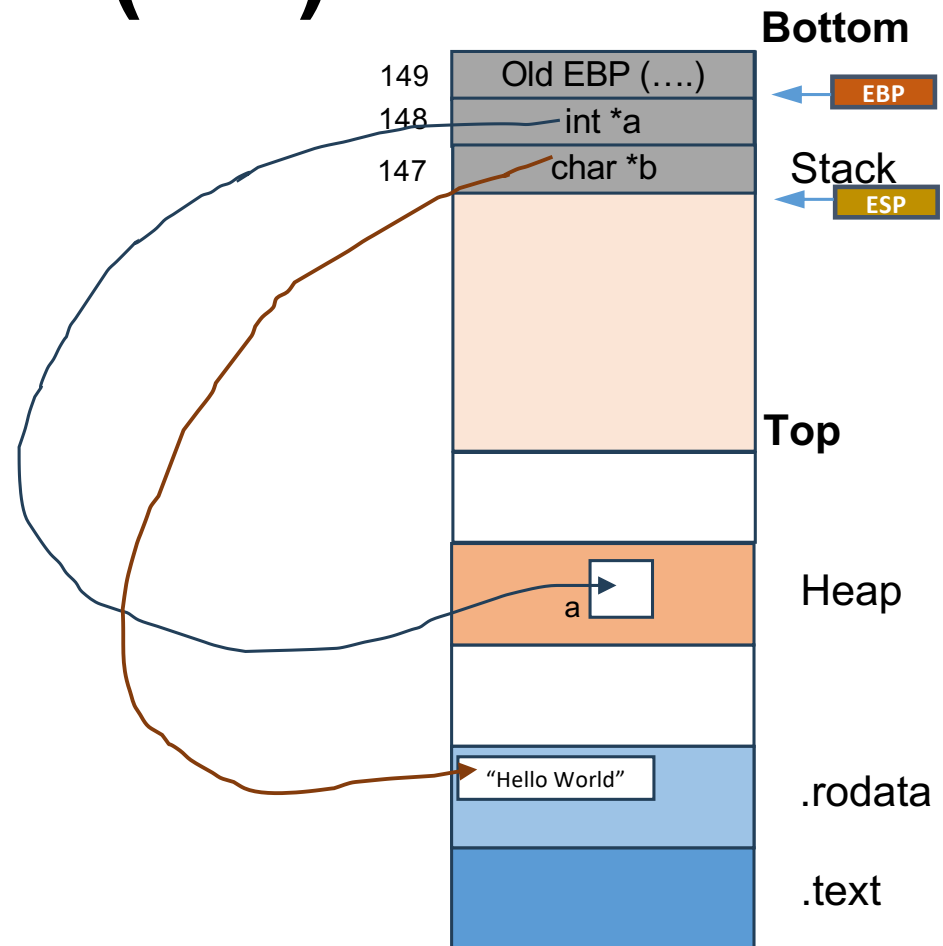
L6: g=*a;

L7: }

L8: void **foo**(int* c) {

L9: *c = 20;

L10: }



Program Execution (4/6)

L1: int g=0;

L2: void **main**() {

L3: int *a = (int*) malloc(4);

L4: char *b = "Hello World";

L5: **foo**(a);

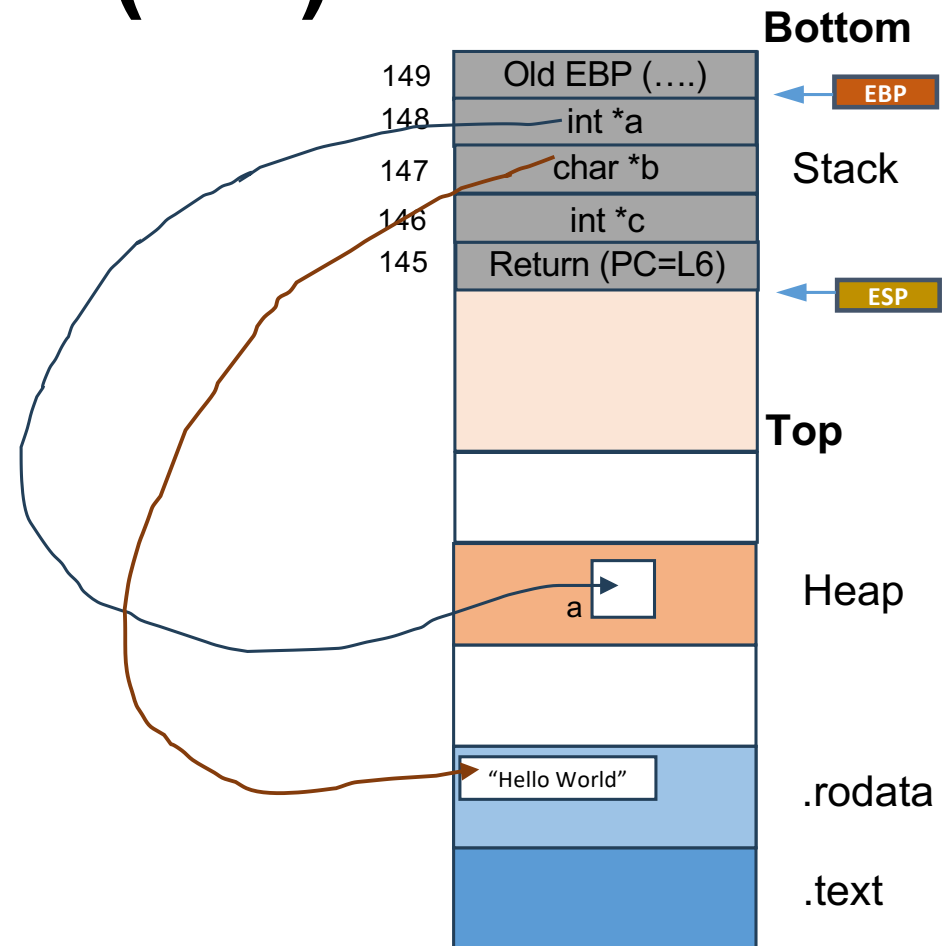
L6: g=*a;

L7: }

L8: void **foo**(int* c) {

L9: *c = 20;

L10: }



Program Execution (5/6)

L1: int g=0;

L2: void **main**() {

L3: int *a = (int*) malloc(4);

L4: char *b = "Hello World";

L5: **foo**(a);

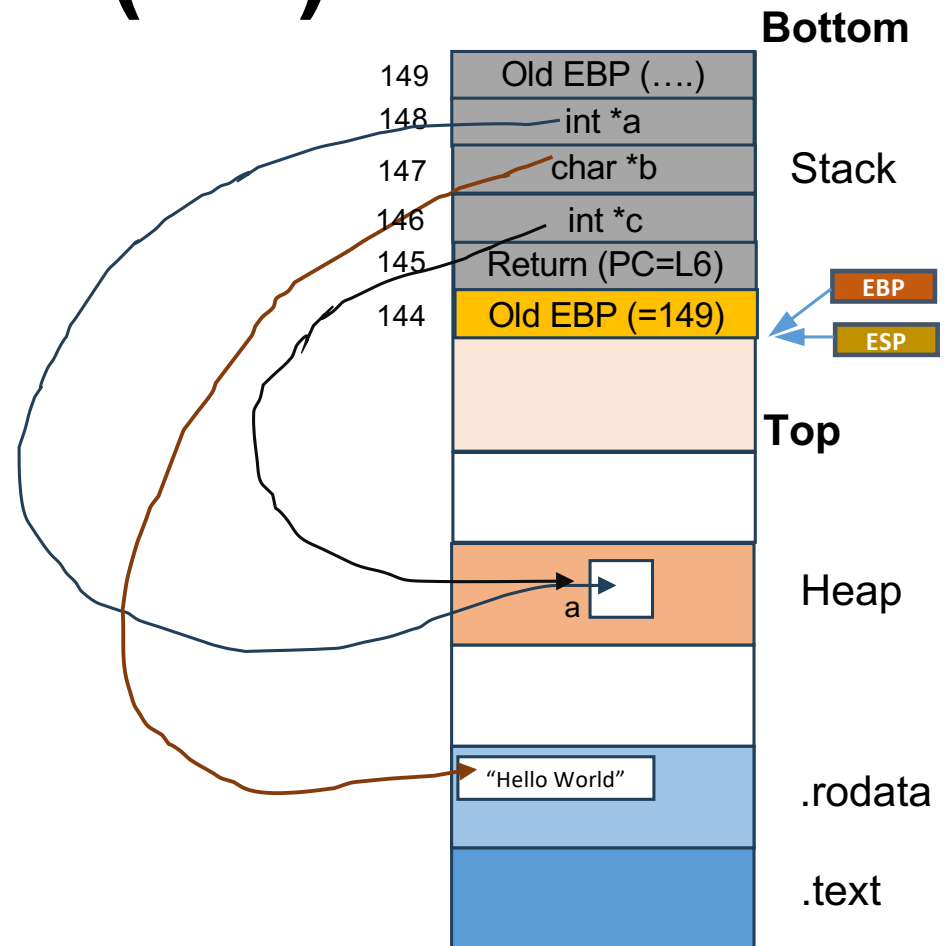
L6: g=*a;

L7: }

L8: void **foo**(int* c) {

L9: *c = 20;

L10: }



Program Execution (6/6)

L1: int g=0;

L2: void **main**() {

L3: int *a = (int*) malloc(4);

L4: char *b = "Hello World";

L5: **foo**(a);

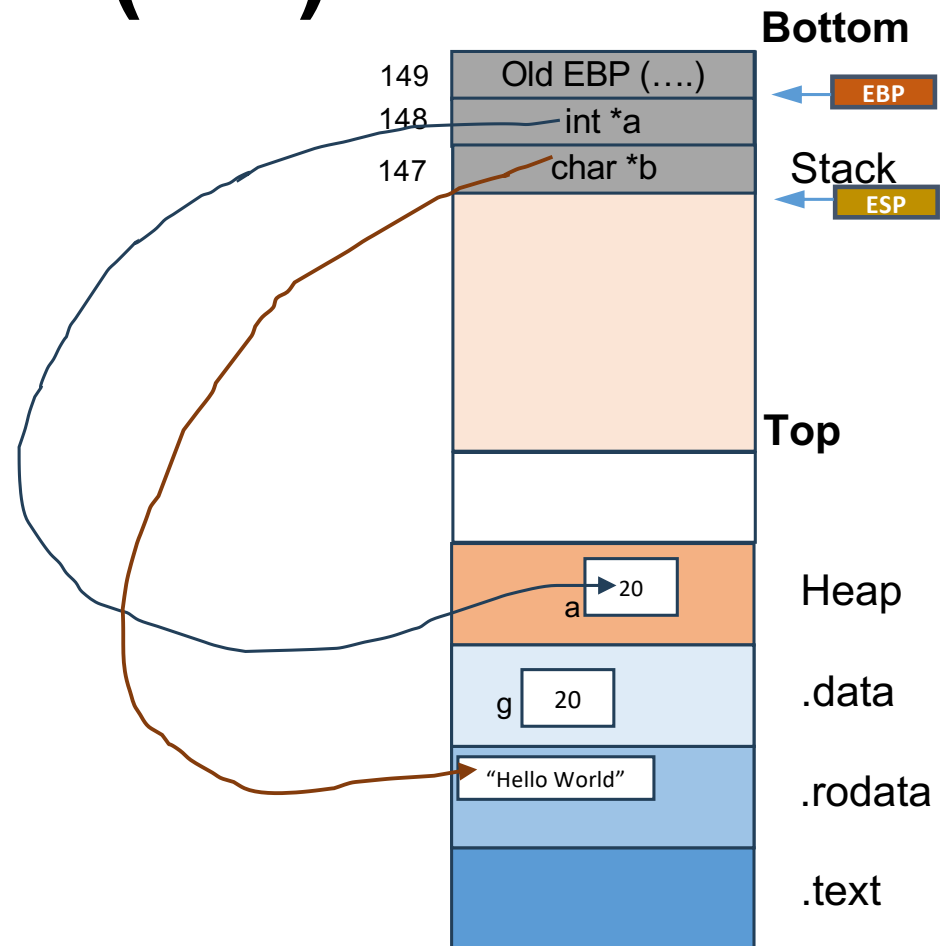
L6: g=*a;

L7: }

L8: void **foo**(int* c) {

L9: *c = 20;

L10: }



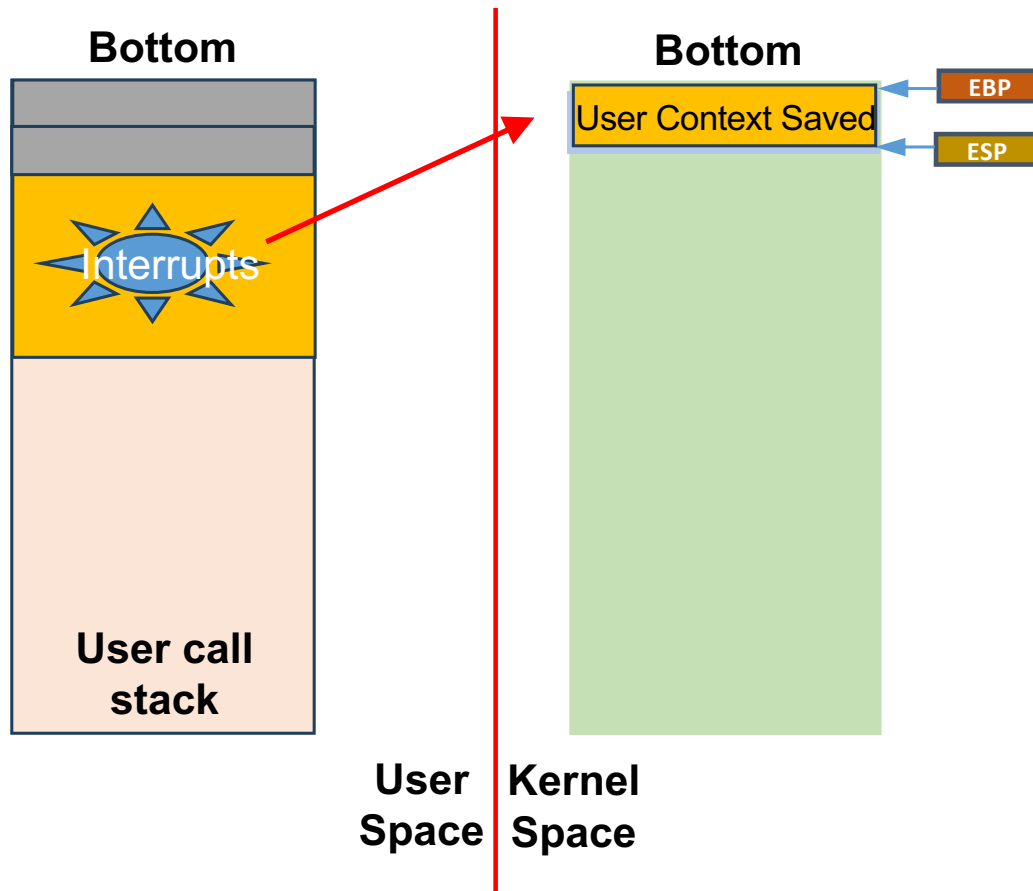
Process State: Example

Time	Process ₀	Process ₁
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		

Process ₀	Process ₁
→ Execute (3 units)	→ Execute (5 units)
→ read() (3 units)	return
→ Execute (2 units)	
return	

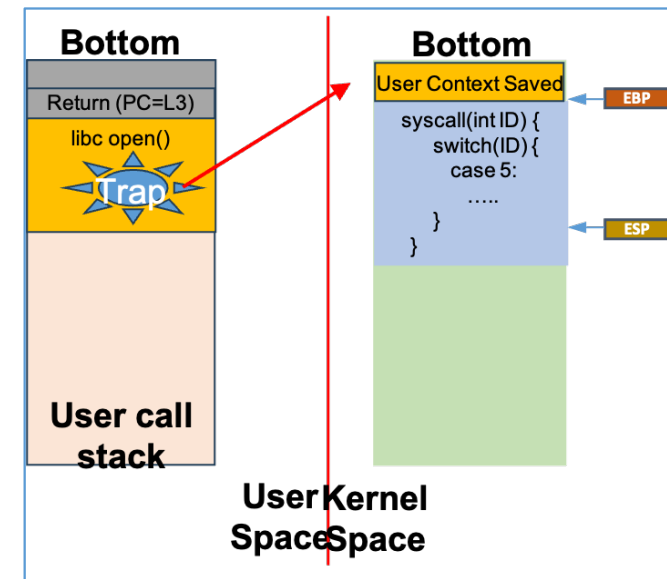
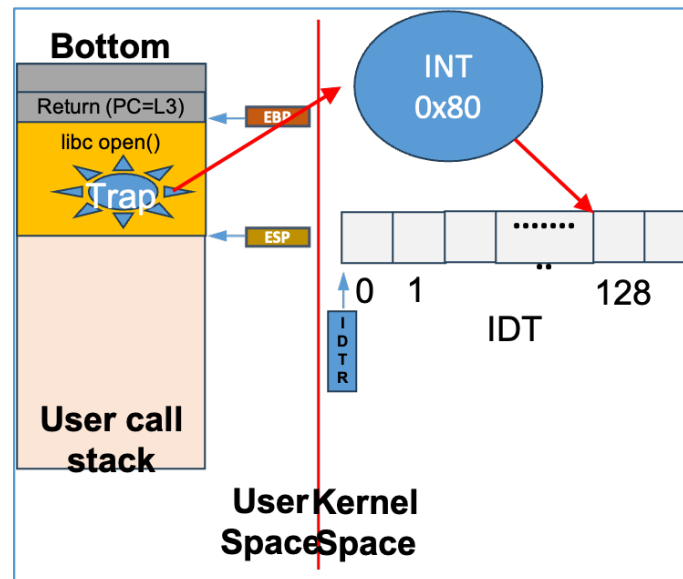
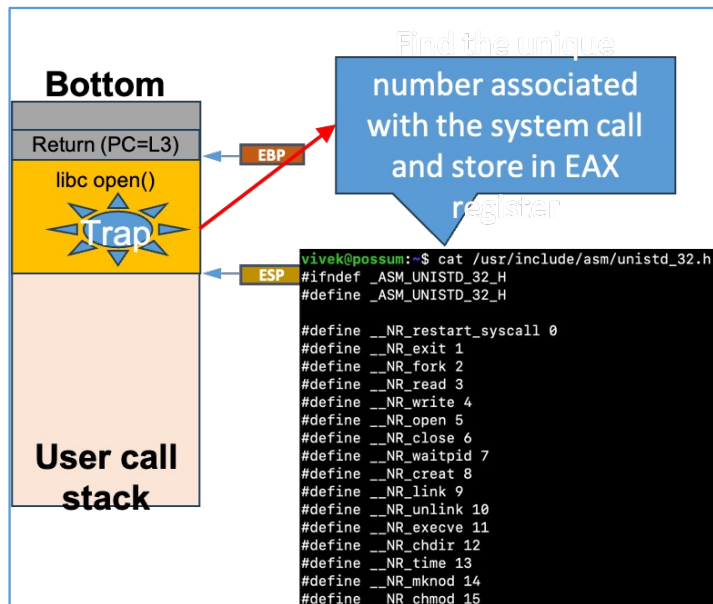
Figure 4.4: Tracing Process State: CPU and I/O

Two Stacks for Each Processes



- OS maintains two stacks for each processes
 - User stack
 - Kernel stack
- Save user stack context (registers) in kernel mode stack while switching to kernel stack from user stack
 - Restore while jumping back to user stack from kernel stack

Steps for Making a System Call



Process Creation & Termination

```
int global=0;
int main() {
    if(fork() == 0) {
        global++;
        printf("Child process: Global=%d\n", global);
        char* args[2] = {"/fib", "40"};
        execv(args[0], args);
        printf("I should never print\n");
    } else {
        printf("I am the Parent process\n");
        int status;
        wait(&status);
    }
    printf("Global=%d\n", global);
    return 0;
}
```

IPC Using Pipes

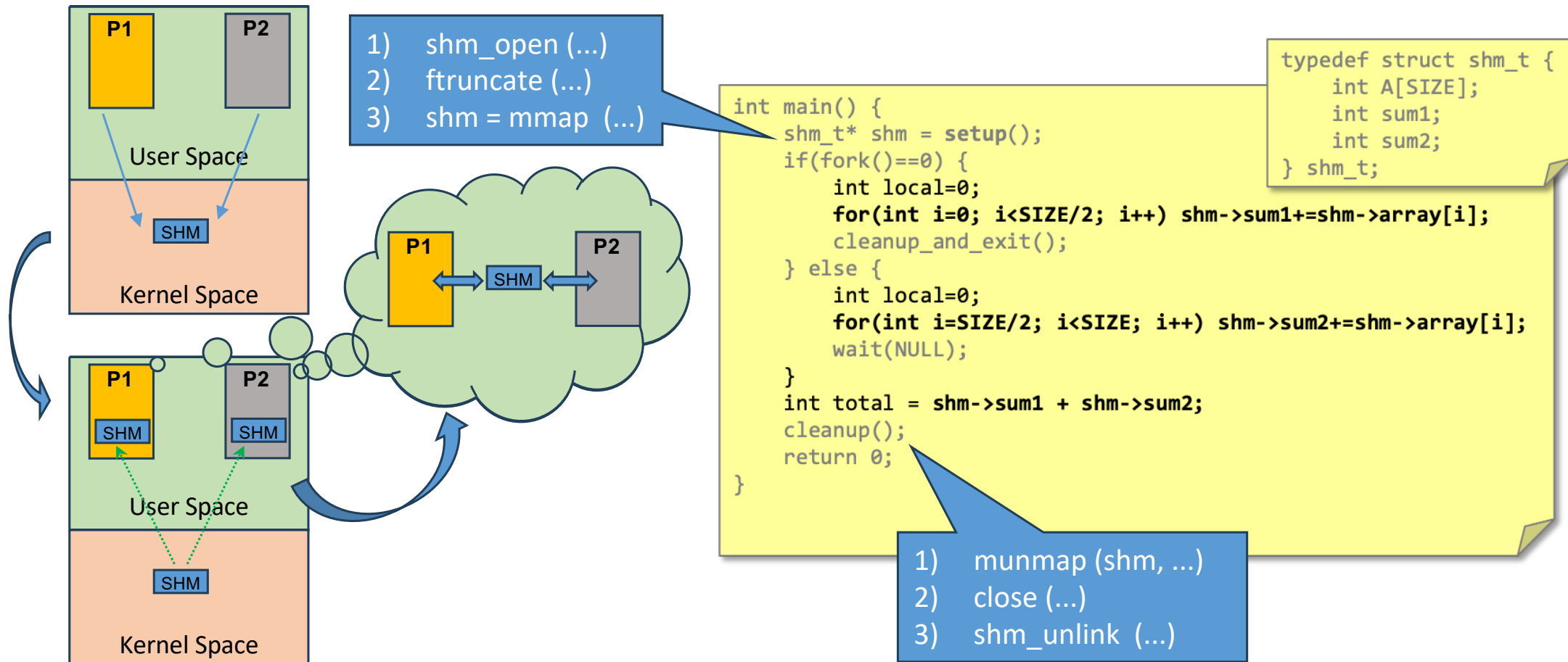
```
int main() {
    int fd[2], status;
    pipe(fd);
    if(fork() == 0) {
        /* Child process */
        close(fd[0]);
        char buff[] = "Hello my dear good Parent";

        write(fd[1], buff, sizeof(buff));
        exit(0);
    }
    /* Parent process */
    close(fd[1]);
    char buff[100];
    read(fd[0], buff, sizeof(buff));
    printf("My obedient child says: %s\n", buff);
    wait(NULL);
    return 0;
}
```

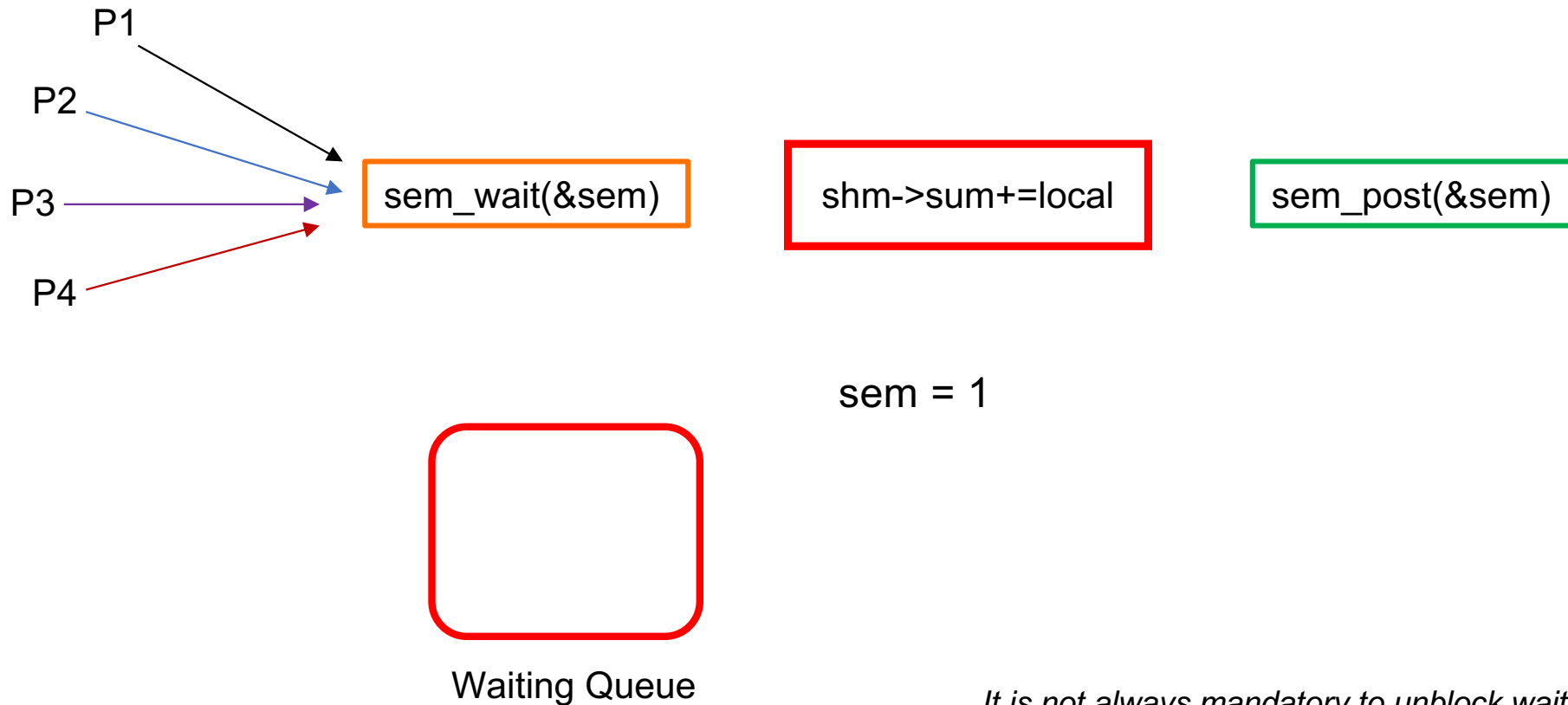


- Pipes (analogous to water pipe) is a unidirectional stream of data flowing from a source process to a destination process
 - Kernel buffer exposed to processes as a pair of file descriptors (readable end and writable end)
 - Data delivered in the same order as sent
 - Uses blocking IO and the writer process will block if the pipe is full
- We use it frequently on Shell
 - Better than using temporary files as pipes automatically clean up themselves unlike files that must be explicitly removed using the “rm” command on Shell

IPC Using POSIX Shared Memory

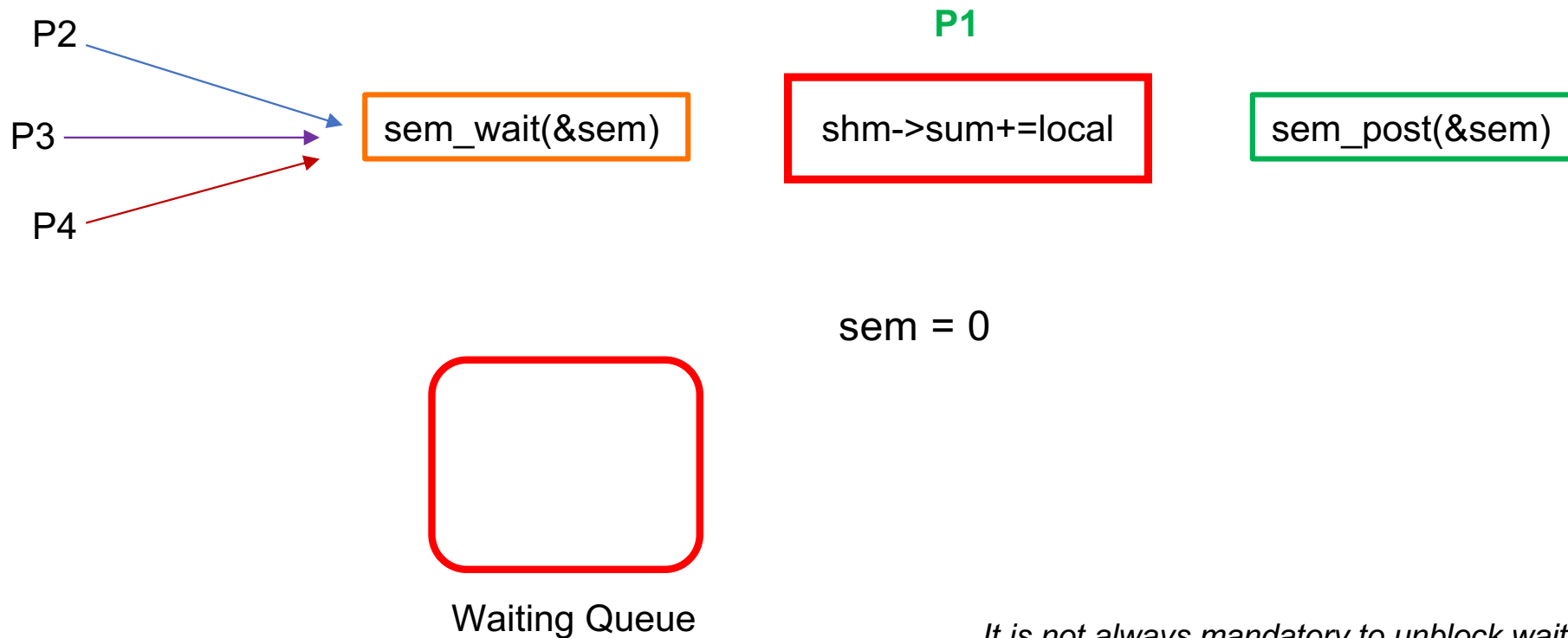


Semaphores (1/6)



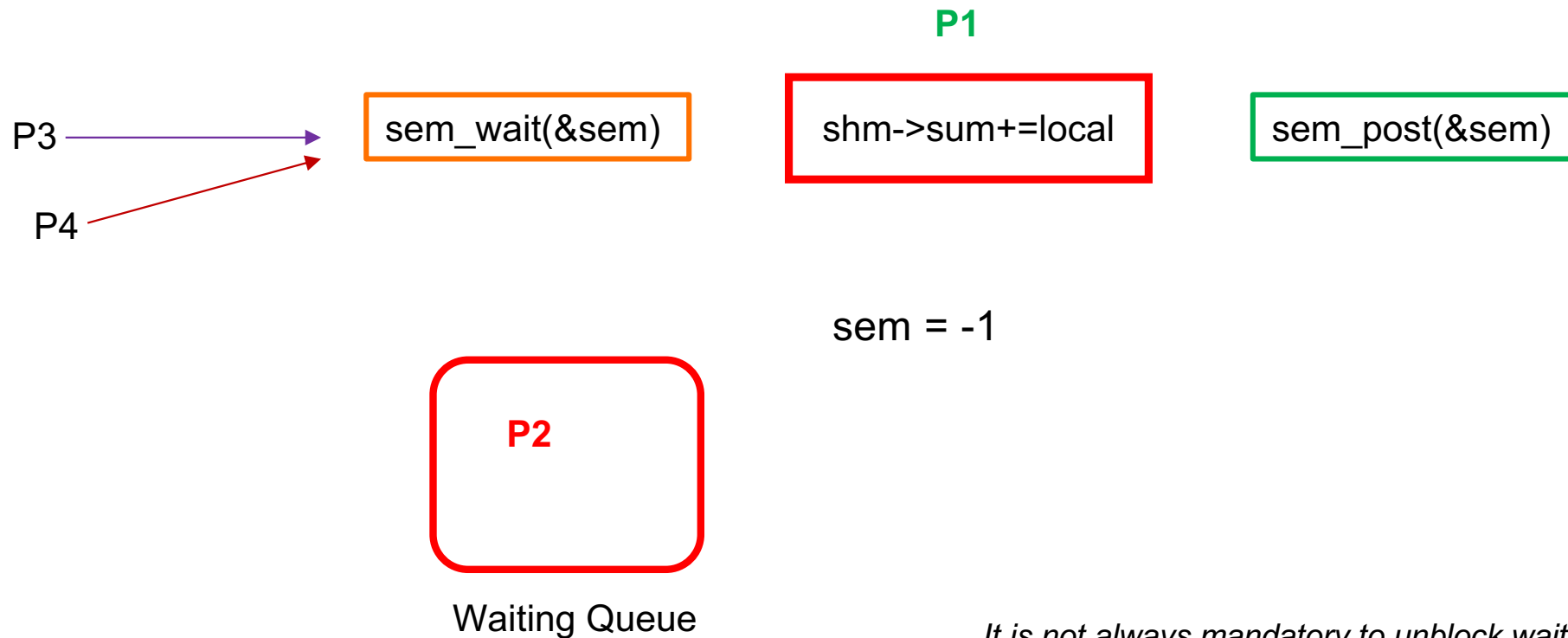
It is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (2/6)



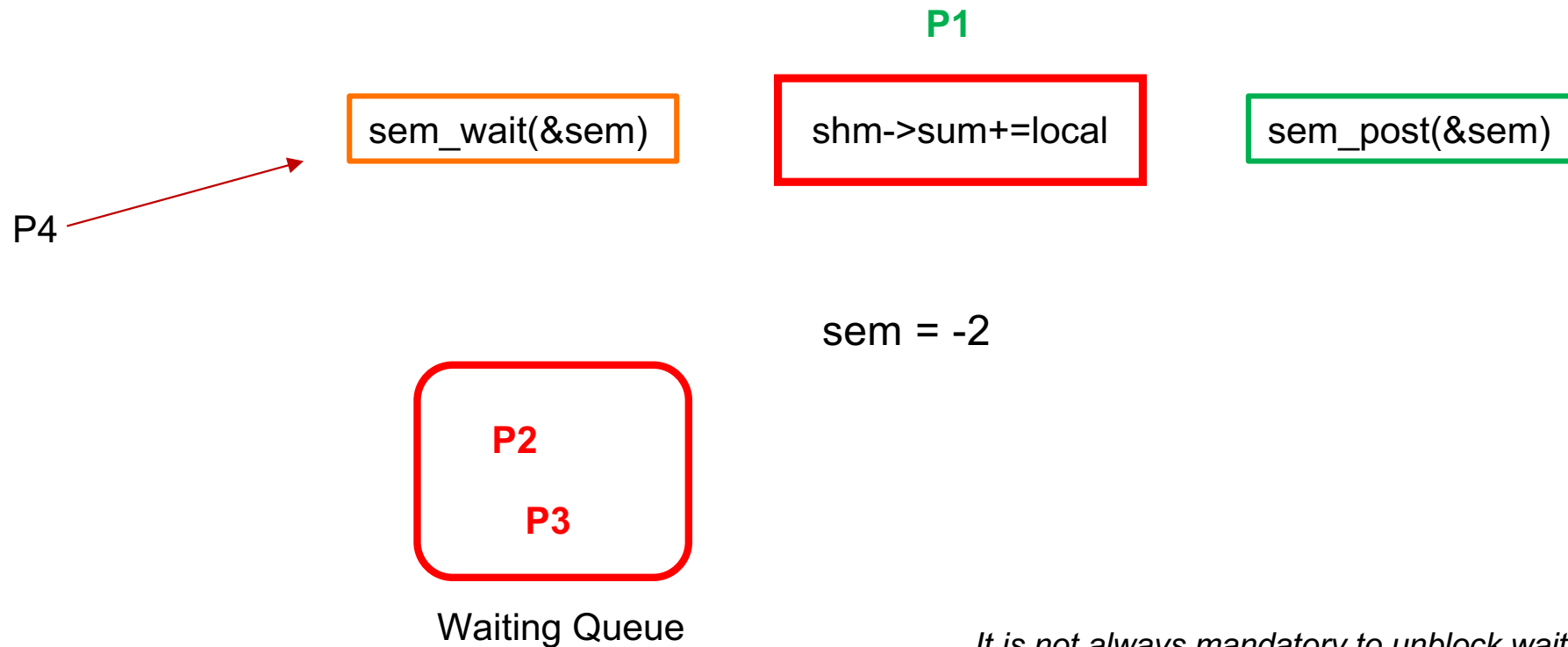
It is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (3/6)



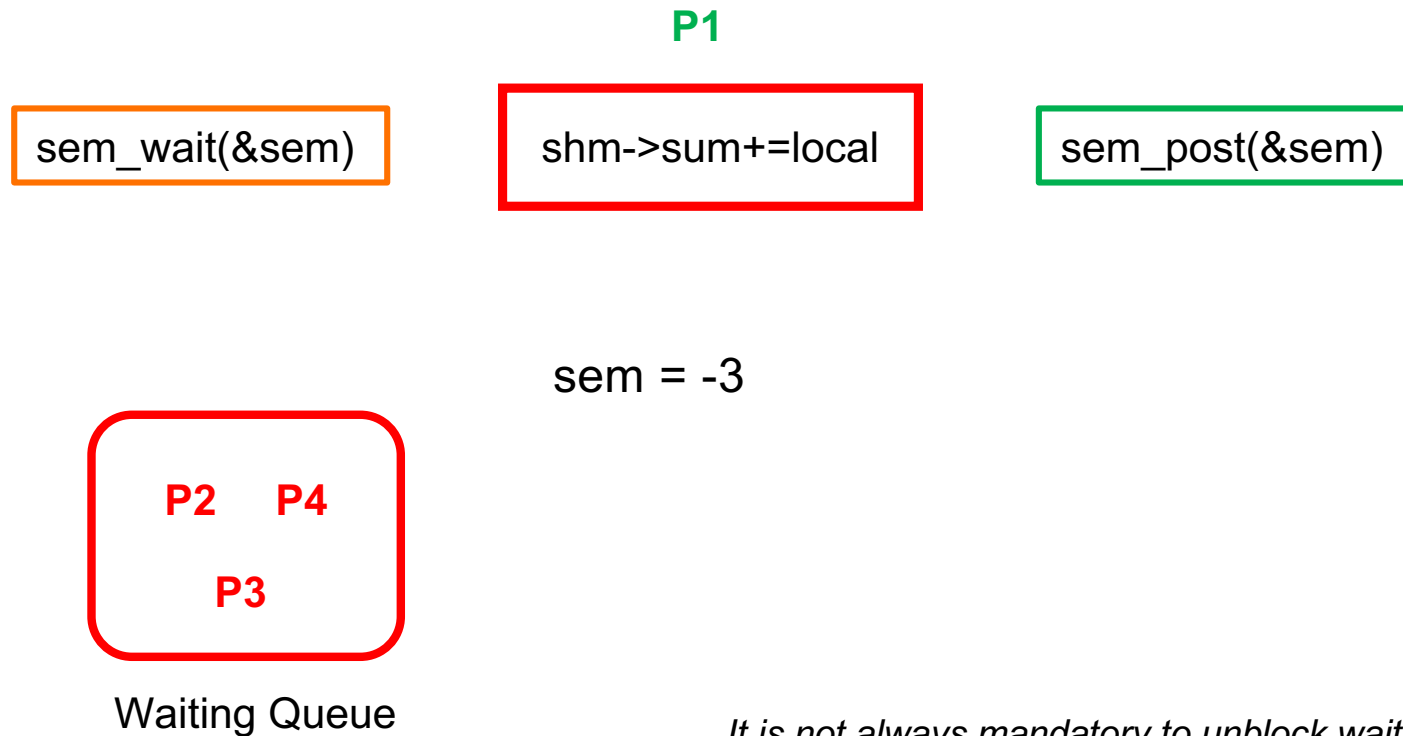
It is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (4/6)



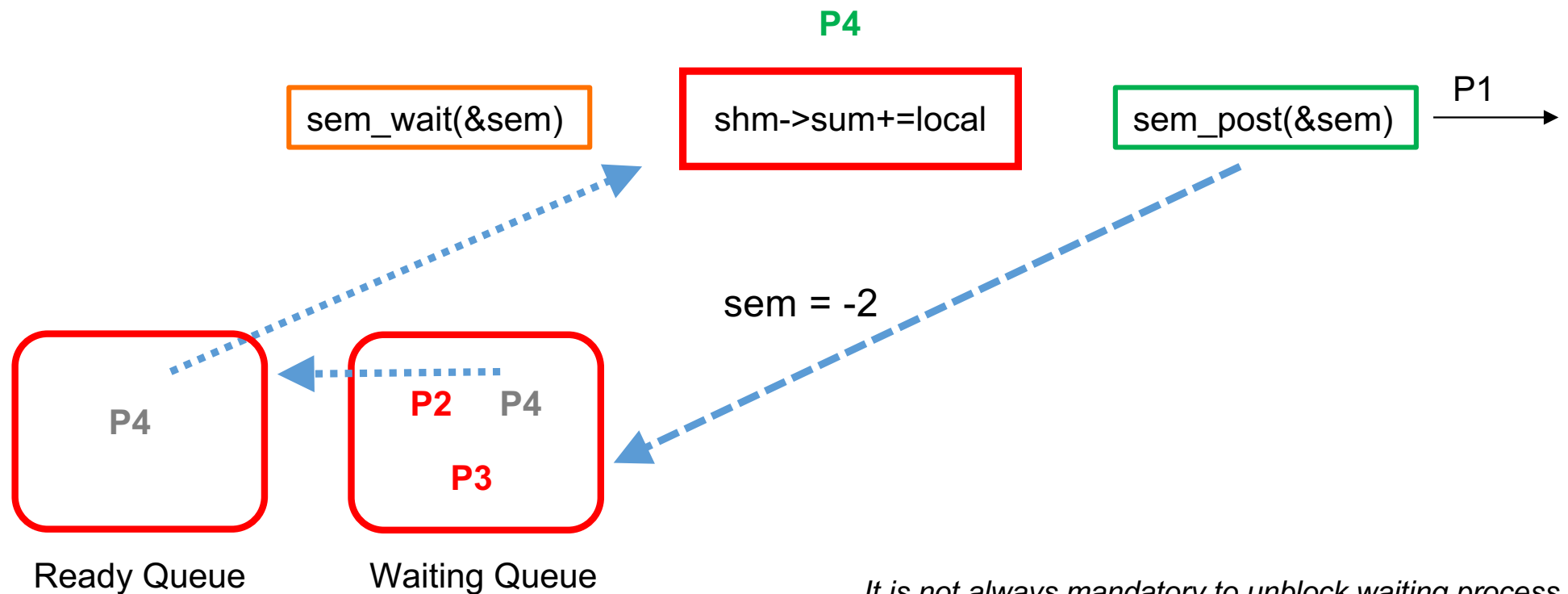
It is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (5/6)



It is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Semaphores (6/6)



It is not always mandatory to unblock waiting process in FIFO order. Some implementations can wake up in unspecified order

Producer Consumer Problem

```
typedef struct cookiejar_t {
    int cookie;
    sem_t jar_empty;
    sem_t jar_full;
} cookiejar_t;

cookiejar_t* cookiejar;

int main() {
    cookiejar = setup();
    cookiejar->empty=1;
    sem_init(&cookiejar->jar_empty, 1, 1);
    sem_init(&cookiejar->jar_full, 1, 0);
    if(fork() == 0) { homer(); }
    if(fork() == 0) { marge(); }
    wait(NULL); // wait for Homer process
    wait(NULL); // wait for Marge process
    sem_destroy(&cookiejar->jar_empty);
    sem_destroy(&cookiejar->jar_full);
    cleanup();
    return 0;
}
```

```
void homer() {
    for(int i=0; i<5; i++) {
        sem_wait(&cookiejar->jar_full);
        printf("Homer ate Cookie-%d\n", cookiejar->cookie);
        sem_post(&cookiejar->jar_empty);
    }
    cleanup_and_exit();
}
```

```
void marge() {
    for(int i=0; i<5; i++) {
        sem_wait(&cookiejar->jar_empty);
        printf("Marge bake Cookie-%d\n", ++cookiejar->cookie);
        sem_post(&cookiejar->jar_full);
    }
    cleanup_and_exit();
}
```

Homer Process (CPU 0)		Sem (empty)	Sem (full)	Marge Process (CPU 1)	
Action	Process State	Sem Value	Sem Value	Action	Process State
Forked	Ready	1	0	Forked	Ready
wait(full)	Blocked	0	-1	wait(empty)	Running
	Blocked	0	-1	Bake Cookie-1	Running
	Ready	0	0	post(full)	Running
Eat Cookie-1	Running	-1	0	wait(empty)	Blocked
post(empty)	Running	0	0		Ready
wait(full)	Blocked	0	-1	Bake Cookie-2	Running
	Ready	0	0	post(full)	Running
.....					

First-Come, First-Served Scheduling

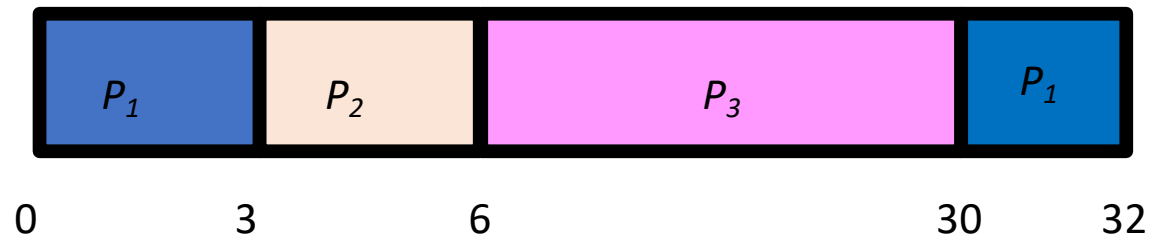
Process

Timeline

P_1 3 (CPU) $\rightarrow$ 2 (I/O) $\rightarrow$ 2 (CPU)

P_2 3 (CPU)

P_3 24 (CPU)



Where are we as of now

● CSE231 Post Conditions



1. Students are able to create a Unix shell with complete clarity about process creation and process execution
2. Students are able to write multi-threaded applications with synchronization primitives and ability to analyze effects of concurrency on process execution and correctness
3. Students are able to analyze the impact of OS concepts, e.g. virtual memory, concurrency, on program execution and ability to fine-tune the program to run efficiently on a given OS
4. Students are able to demonstrate deeper understanding of the Unix-like OSes and kernel programming

All the best for your exam !!

- Exam syllabus
 - Lectures **02-11**
- Exam schedule
 - Date: Monday (27nd September)
 - Time: 10:00am-11:00am (One hour only)
- Exam pattern
 - MCQ (similar to quizzes)
 - Subjective questions
 - Program writing